Week 2 Research Report

Topics Covered

1. JavaScript Form Validator
2. Git Fetch vs Git Pull
3. JavaScript Map, Filter, and Reduce
4. Fetch API vs Axios
5. Vanilla JavaScript Form Validation

Prepared by

Siddharth Puhan

Intern Trainee

1. JavaScript Form Validator

What is form validation?

Form validation checks that the data a user types into a form is complete and correct **before** it is used or sent to a server.

Why is it needed?

- It catches mistakes early, such as an empty name or an email without "@".
- It gives the user instant feedback, without waiting for the server.
- It keeps the collected data clean and consistent.
- It is a convenience only. The server must always validate the data again for security.

How JavaScript validates input

- Read the value with `input.value` and remove extra spaces with `.trim()`.
- Empty check: `value === ""`.
- Length check: `value.length < 3`.
- Pattern check: a regular expression with `regex.test(value)` (for example, an email format).

Showing error and success messages

- Add an empty `<span>` next to the field to hold the message.
- Set its `textContent` to an error or a success message, and change its colour to match.
- Clear or replace the old message each time the check runs.

Example: validating a username field

Rules: the field is required and must have at least 3 characters.

```
<input type="text" id="username" placeholder="Username">
<span id="msg"></span>
<button type="button" onclick="checkUsername()">Check</button>

<script>
function checkUsername() {
  const value = document.getElementById("username").value.trim();
  const msg = document.getElementById("msg");

  if (value === "") {
    msg.textContent = "Username is required.";
    msg.style.color = "red";
  } else if (value.length < 3) {
    msg.textContent = "Use at least 3 characters.";
    msg.style.color = "red";
  } else {
    msg.textContent = "Looks good!";
    msg.style.color = "green";
  }
}
</script>
```

How it works: the function reads and trims the value, tests each rule in order, and shows the first error it finds. If no rule fails, it shows a success message.

2. Git Fetch vs Git Pull

Both commands get updates from a remote repository (such as GitHub). They differ in what happens after the download.

What does git fetch do?

- Downloads new commits, branches, and tags from the remote repository.
- Updates remote-tracking branches such as `origin/main`.
- Does **not** change your working files or your current branch, so it is safe to run at any time.

What does git pull do?

- Runs `git fetch` and then merges the downloaded changes into your current branch.
- `git pull --rebase` fetches and then rebases your commits instead of merging.
- It can change your files and may cause merge conflicts.

Key differences

Point	git fetch	git pull
What it does	Downloads changes only	Downloads and merges changes
Changes your files?	No	Yes
Updates your current branch?	No	Yes
Risk of conflicts	None while fetching	Possible
Equivalent to	-	git fetch + git merge

When to use each command

- **Use git fetch** when you want to see what changed before merging, check whether teammates pushed new work, or review updates before rebasing.
- **Use git pull** when your working tree is clean and you simply want the latest changes right away.

Terminal examples

```
# Download updates without touching your files
git fetch origin

# See which commits are new on the remote branch
git log HEAD..origin/main --oneline

# Compare your branch with the remote branch
git diff main origin/main

# Merge the fetched changes when you are ready
git merge origin/main

# Fetch and merge in a single step
git pull origin main

# Fetch and rebase instead of merge
git pull --rebase origin main
```

Tip: commit or stash your local changes before running `git pull`. Older repositories may use `master` instead of `main`.

3. JavaScript Map, Filter, and Reduce

These are array methods. None of them change the original array, and they all take a function that runs on each item. We use this array in every example:

```
const numbers = [1, 2, 3, 4, 5];
```

map(): change every item

`map()` runs a function on each item and returns a **new array of the same length**.

```
const doubled = numbers.map(n => n * 2);
console.log(doubled);   // Output: [2, 4, 6, 8, 10]
```

filter(): keep only some items

`filter()` keeps the items for which the function returns **true**. The new array can be shorter.

```
const evens = numbers.filter(n => n % 2 === 0);
console.log(evens);     // Output: [2, 4]
```

reduce(): combine items into one value

`reduce()` goes through the array and builds a single result. It takes a function with **total** (the running result) and the **current item**, plus a starting value (here `0`).

```
const sum = numbers.reduce((total, n) => total + n, 0);
console.log(sum);       // Output: 15
```

Comparison

Method	Use it to...	Returns	Length of result
map()	Transform every item	A new array	Same as original
filter()	Select items that pass a test	A new array	Same or shorter
reduce()	Combine items into one value	Any single value	Not an array (usually)

Using them together

Methods can be chained. This example squares the even numbers and adds them up:

```
const result = numbers
  .filter(n => n % 2 === 0)        // [2, 4]
  .map(n => n * n)                 // [4, 16]
  .reduce((total, n) => total + n, 0);
console.log(result);               // Output: 20
```

4. Fetch API vs Axios

What are they?

- **Fetch API:** a built-in JavaScript feature for making HTTP requests. It works in modern browsers and in Node.js 18 and later. No installation is needed.
- **Axios:** a popular third-party library for HTTP requests. Install it with `npm install axios`.

Both send a request to a URL (GET, POST, etc.) and return a Promise with the server's response.

GET request with Fetch

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(response => {
    if (!response.ok) {
      throw new Error("HTTP error: " + response.status);
    }
    return response.json();          // convert the body to a JS object
  })
  .then(data => console.log(data.name))
  .catch(error => console.error(error.message));
```

GET request with Axios

```
import axios from "axios";

axios.get("https://jsonplaceholder.typicode.com/users/1")
  .then(response => console.log(response.data.name))   // data is already parsed
  .catch(error => console.error(error.message));
```

Main differences

- **Syntax:** Fetch needs a separate step to read the body. Axios gives you the data directly.
- **Response handling:** Fetch uses `response.json()`. Axios puts the parsed data in `response.data`.
- **Error handling:** Fetch only fails on network problems. A 404 or 500 does **not** throw, so you must check `response.ok`. Axios rejects automatically for any status outside 200-299, and the status is in `error.response.status`.

Feature	Fetch API	Axios
Availability	Built in, no install	Needs `npm install axios`
Parsing JSON	Manual: `response.json()`	Automatic: `response.data`
HTTP error (404, 500)	No error thrown; check `response.ok`	Promise is rejected automatically
Sending a JSON body	`JSON.stringify()` and headers	Pass the object directly
Timeout	Use AbortController	Built-in `timeout` option
Best for	Simple requests, small projects	Larger apps, repeated API calls

5. Vanilla JavaScript Form Validation

Vanilla JavaScript means plain JavaScript with no libraries or frameworks (no jQuery, React, or validation packages). It runs in every browser. Page 1 covered the general idea; here we focus on the implementation.

Key building blocks

- `addEventListener("submit", fn)` runs your function when the form is submitted.
- `event.preventDefault()` stops the default behaviour (page reload and sending data) so you can check the input first.
- **Input checks:** read each value, compare it with your rules, show an error if a rule fails, and track the result in a variable such as `isValid`.
- Add `novalidate` to the form so the browser's own pop-ups do not hide your messages.

Example: sign-up form with a success message

```
<form id="signup" novalidate>
  <input type="text" id="name" placeholder="Name"> <small id="nameError"></small>
  <input type="email" id="email" placeholder="Email"> <small id="emailError"></small>
  <input type="password" id="password" placeholder="Password">
  <small id="passwordError"></small>
  <button type="submit">Sign Up</button>
  <p id="success"></p>
</form>
```

```
const form = document.getElementById("signup");

function setText(id, message) {
  document.getElementById(id).textContent = message;
}

form.addEventListener("submit", function (event) {
  event.preventDefault();                        // stop the page reload
  const name = document.getElementById("name").value.trim();
  const email = document.getElementById("email").value.trim();
  const password = document.getElementById("password").value;
  let isValid = true;

  setText("nameError", "");  setText("emailError", "");
  setText("passwordError", "");  setText("success", "");

  if (name === "") {
    setText("nameError", "Name is required.");
    isValid = false;
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    setText("emailError", "Enter a valid email address.");
    isValid = false;
  }
  if (password.length < 8) {
    setText("passwordError", "Use at least 8 characters.");
    isValid = false;
  }
  if (isValid) {
    setText("success", "Form submitted successfully!");
    form.reset();
  }
});
```

Best practices

- Always validate again on the server. Browser checks can be bypassed.
- Write clear messages that tell the user how to fix the problem, and show them next to the field.
- Trim spaces, and use the right `type` (such as `email`) for each input.
- Do not rely on colour alone; always include text in the message.